BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 1

Roll no:53 PRN No:12414903 Name of Student: Sara Ansingkar

Subject Name : Deep Learning

Problem Statement:

Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test splitting, and data visualization on a sample dataset.

Algorithm:

1. Start
2. Import Libraries of TensorFlow and matplotlib
3. Download dataset from: https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
4. Perform data preprocessing by scaling down values of pixels from 0-255 to 0-1
5. Display the dataset shape and sample image
6. Create a **Sequential Neural Network** mode with ReLu and Softmax activation
7. Train the model for 3 epochs
8. Evaluate the model and display its test accuracy
9. Save the trained model as **my_mnist_model.keras**.
10. Stop

Code and Output:

```
[1]
✓ 5s
import tensorflow as tf
import matplotlib.pyplot as plt
```

```
[3]
✓ 0s
(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.mnist.load_data()
```

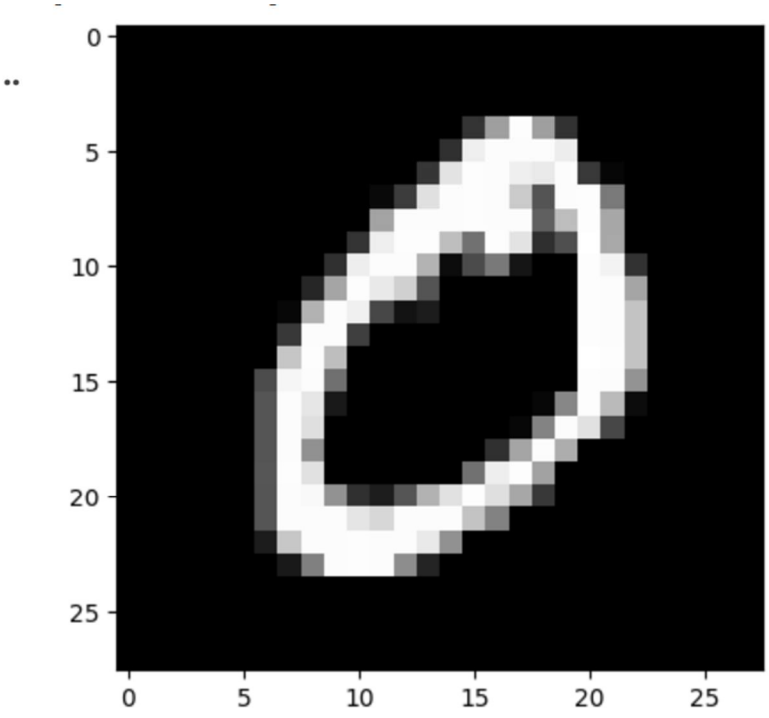
```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
[4]
✓ 0s
# Scale down values of pixels from 0-255 to 0-1
train_images = train_images /255.0
test_images = test_images / 255.0
```

```
[5]
✓ 0s
# Visualizing data
print(train_images.shape)
print(test_images.shape)
print(train_labels)

# Display first image
plt.imshow(train_images[1], cmap='gray')
plt.show()
```

```
(60000, 28, 28)
(10000, 28, 28)
[5 0 4 ... 5 6 8]
```



[6] ✓ 22s

```
# Defining neural network model
my_model = tf.keras.models.Sequential()
my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
my_model.add(tf.keras.layers.Dense(128, activation='relu'))     #Rectified Linear Unit        #Dense layers are used to learn complex patter
my_model.add(tf.keras.layers.Dense(10, activation='softmax'))   #softmax is often used as the activation function in the output layer for cl

# Compiling the model
my_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy']) #Adaptive Moment Estimation #sparse_categor

# Training the model
my_model.fit(train_images, train_labels, epochs=3)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim` argum
  super().__init__(**kwargs)
Epoch 1/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 3ms/step - accuracy: 0.9258 - loss: 0.2607
Epoch 2/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9669 - loss: 0.1152
Epoch 3/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9763 - loss: 0.0786
<keras.src.callbacks.history.History at 0x7d3e88f44e60>
```

[7] ✓ 0s

```
# Checking model accuracy on test data
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print('Test accuracy: ', val_acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9754 - loss: 0.0809
Test accuracy:  0.9753999710083008
```

[9] ✓ 0s

```
# Saving the model
my_model.save('my_mnist_model.keras')
```